Beagle Jacky-Anaïs	BTS SIO — Option SISR	Projet E6 — Proxy Squid / cab1
Année 2025-2026	Conteneur cab1 — 10.63.101.70	Mars 2026

Projet E6 — Proxy filtrant Squid dans un conteneur systemd-nspawn

Authentification et filtrage web — Conteneur cab1 — BTS SIO SISR

Contexte et objectif

Dans le cadre du BTS SIO option SISR, ce projet est réalisé sur le réseau réel de l'établissement (10.0.0.0/8) en salle E63. L'objectif est de déployer un proxy filtrant Squid à l'intérieur d'un conteneur systemd-nspawn (cab1), hébergé sur le PC2 Debian 12 (anabea@am-6451063e564c). Le conteneur dispose d'une adresse IP fixe (10.63.101.70), ce qui garantit une adresse proxy stable et accessible depuis tous les postes du réseau, même lorsque les PCs physiques obtiennent leur adresse IP dynamiquement via DHCP.

Le proxy a pour rôle d'authentifier obligatoirement les utilisateurs du réseau et d'appliquer des règles de filtrage différenciées selon le profil : toto bénéficie d'un accès standard avec les réseaux sociaux bloqués, tandis que tata dispose d'un accès restreint avec YouTube et Twitch bloqués en plus. Sans authentification, tout accès est refusé (code HTTP 407). L'ensemble est réalisé en ligne de commande par projet2, superutilisateur du conteneur.

Schéma réseau — Infrastructure lycée

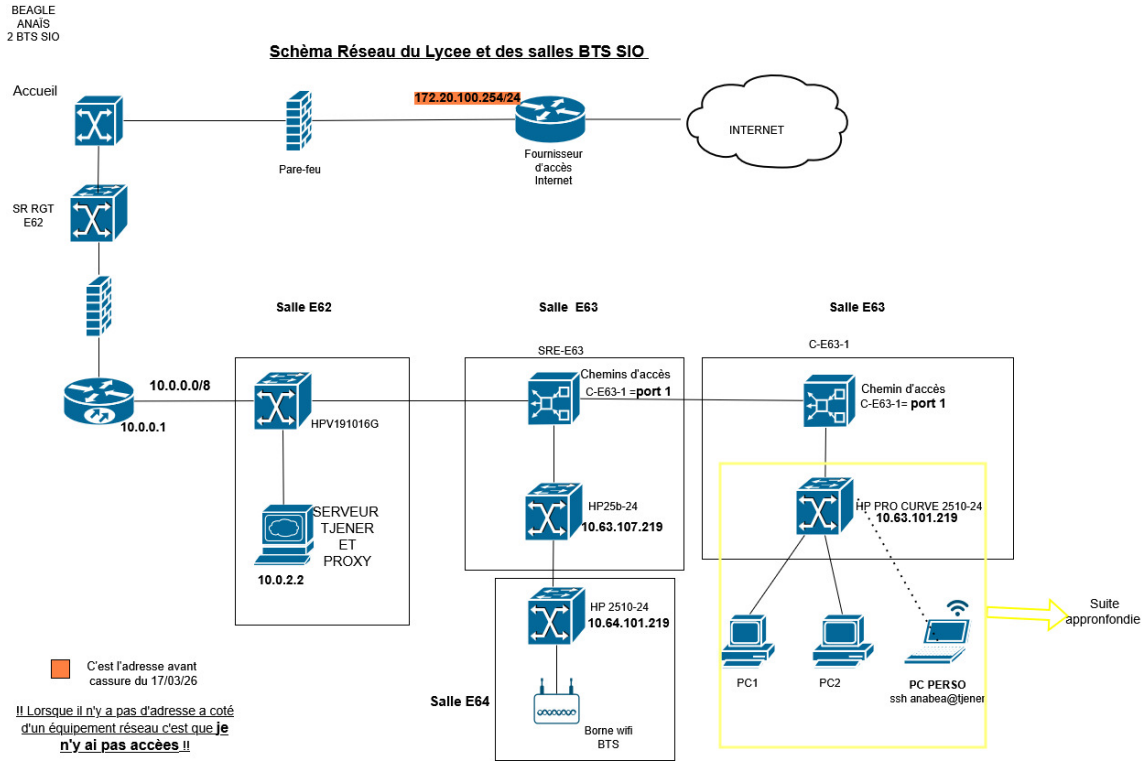
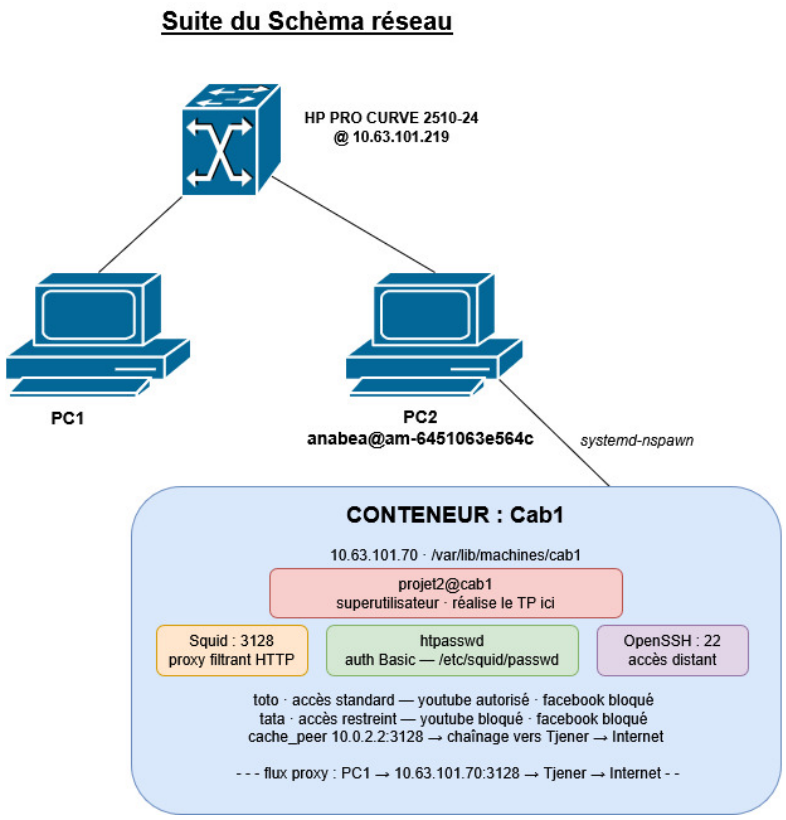


Schéma réseau du lycée — Salle E62 (Tjener/Proxy 10.0.2.2), Salle E63 (PC2 hôte de cab1, Switch HP PRO CURVE 2510-24 @ 10.63.101.219)

Beagle Jacky-Anaïs	BTS SIO — Option SISR	Projet E6 — Proxy Squid / cab1
Année 2025-2026	Conteneur cab1 — 10.63.101.70	Mars 2026

Schéma réseau détaillé — Conteneur cab1



Suite du schéma — PC2 hôte, conteneur cab1 (10.63.101.70) avec projet2 / Squid :3128 / toto / tata, PC1 client

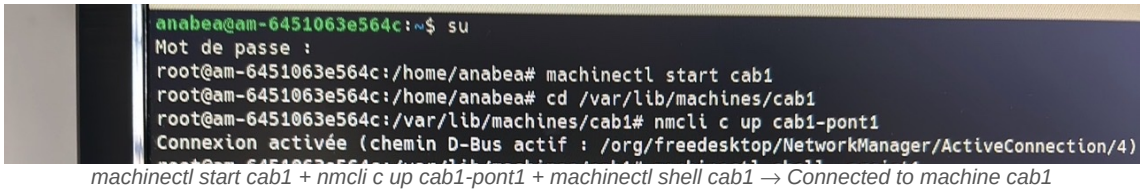
Flux HTTP : Client → Squid cab1 (10.63.101.70:3128) → Tjener (10.0.2.2:3128) → Internet

Beagle Jacky-Anaïs	BTS SIO — Option SISR	Projet E6 — Proxy Squid / cab1
Année 2025-2026	Conteneur cab1 — 10.63.101.70	Mars 2026

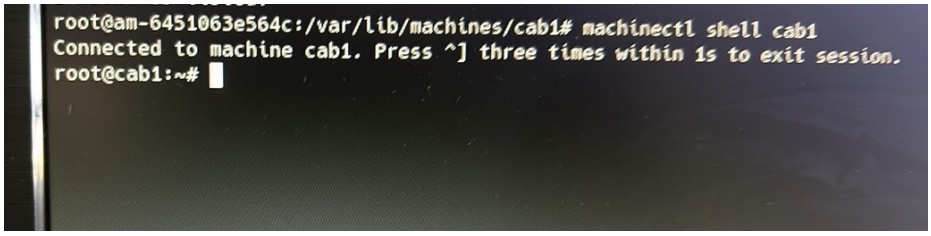
1. Accès au conteneur cab1

Depuis PC2, on se place dans le répertoire des machines virtuelles, on démarre le conteneur, on active son interface réseau puis on y entre via machinectl.

```
cd /var/lib/machines
machinectl start cab1
nmcli c up cab1-pont1
machinectl shell cab1
```



machinectl start cab1 + nmcli c up cab1-pont1 + machinectl shell cab1 → Connected to machine cab1

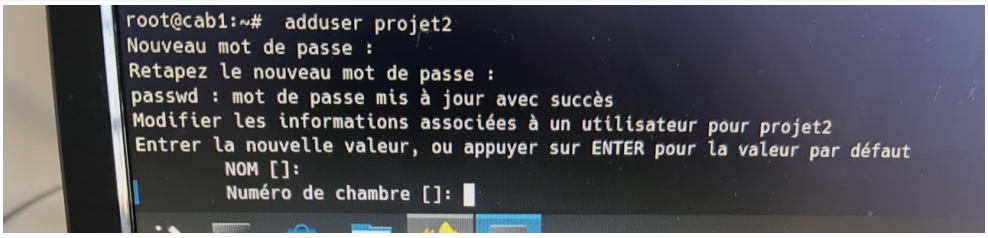


root@cab1:~# — connexion au conteneur cab1 confirmée

2. Création de l'utilisateur projet2 et droits superutilisateur

On crée l'utilisateur projet2 qui réalisera l'ensemble du TP, puis on lui accorde les droits sudo pour l'administration du conteneur.

```
adduser projet2
```



adduser projet2 — création de l'utilisateur projet2 dans cab1

```
usermod -aG sudo projet2
id projet2
```

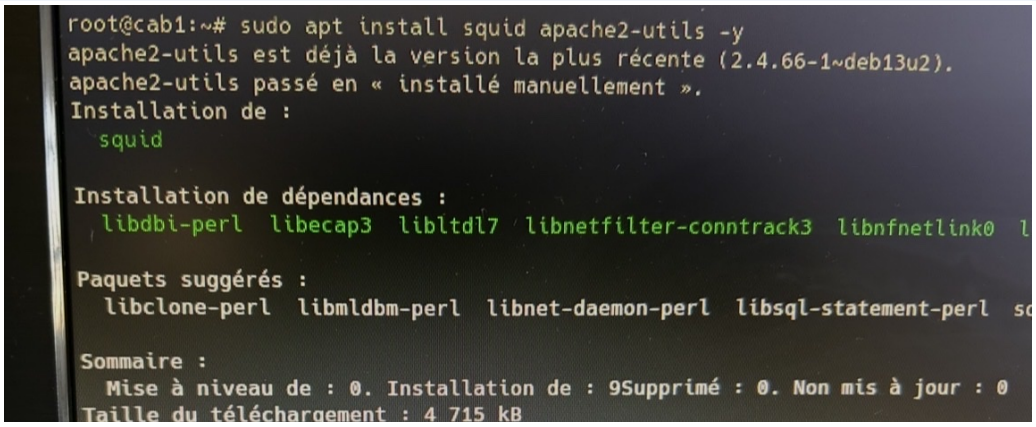
projet2 est maintenant superutilisateur du conteneur — il peut installer des paquets et modifier les configurations système.

3. Installation de Squid, htpasswd et curl

On installe Squid (proxy HTTP), apache2-utils (contient htpasswd pour l'authentification) et curl (pour les tests en ligne de commande).

```
sudo apt install squid apache2-utils -y
```

```
sudo apt install curl -y
```



sudo apt install squid apache2-utils -y — Squid 6.13 installé avec ses dépendances

4. Création des utilisateurs système toto et tata

Les utilisateurs système toto et tata sont créés avec adduser. Ces comptes servent de base pour les tests de filtrage différencié.

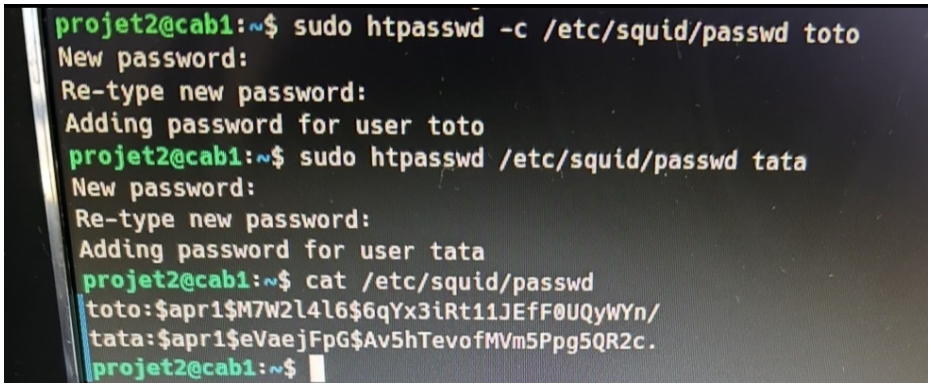
```
adduser toto
```

```
adduser tata
```

5. Création des comptes proxy (htpasswd)

Les identifiants proxy sont stockés dans /etc/squid/passwd. Le flag -c crée le fichier lors de l'ajout du premier utilisateur (toto). Pour tata, on omet -c afin de ne pas écraser le fichier existant.

```
sudo htpasswd -c /etc/squid/passwd toto
sudo htpasswd /etc/squid/passwd tata
cat /etc/squid/passwd
```

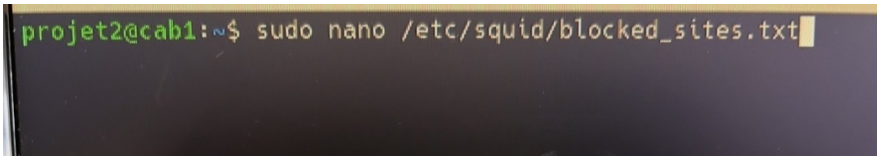


htpasswd -c toto + htpasswd tata + cat /etc/squid/passwd — toto et tata présents avec leurs mots de passe chiffrés

6. Création des listes de sites bloqués

Deux fichiers texte définissent les domaines filtrés. blocked_sites.txt s'applique à tous les utilisateurs. blocked_tata.txt ajoute des restrictions supplémentaires pour tata uniquement.

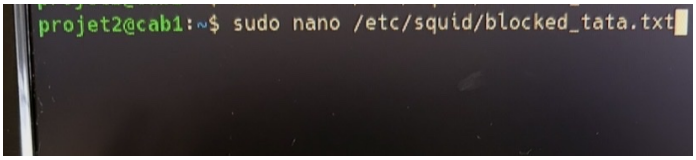
```
sudo nano /etc/squid/blocked_sites.txt
```



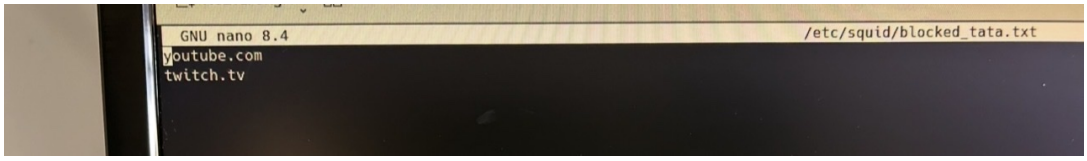
nano /etc/squid/blocked_sites.txt — ouverture du fichier

Contenu de blocked_sites.txt — facebook.com, instagram.com, tiktok.com, x.com

```
sudo nano /etc/squid/blocked_tata.txt
```



nano /etc/squid/blocked_tata.txt — ouverture du fichier

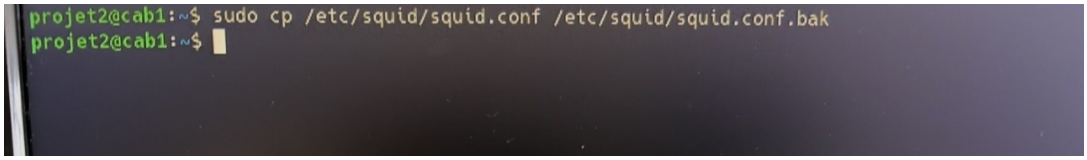


Contenu de blocked_tata.txt — youtube.com, twitch.tv

7. Sauvegarde et configuration de squid.conf

Avant toute modification, on sauvegarde la configuration originale. La configuration définit l'authentification Basic, les ACL par utilisateur, les règles de filtrage et le chaînage vers le Tjener.

```
sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.bak
```



cp squid.conf squid.conf.bak — sauvegarde effectuée avant modification

```
sudo nano /etc/squid/squid.conf
```

Contenu configuré :

```
http_port 3128
auth_param basic program /usr/lib/squid/basic_ncsa_auth /etc/squid/passwd
auth_param basic realm "Proxy BTS — Identifiez-vous"
auth_param basic credentialsttl 2 hours
acl bts_network src 10.0.0.0/8
acl utilisateurs_auth proxy_auth REQUIRED
acl user_tata proxy_auth tata
acl sites_bloques dstdomain "/etc/squid/blocked_sites.txt"
acl sites_bloques_tata dstdomain "/etc/squid/blocked_tata.txt"
http_access deny sites_bloques
http_access deny user_tata sites_bloques_tata
http_access allow utilisateurs_auth bts_network
http_access deny all
cache_peer 10.0.2.2 parent 3128 0 no-query default
never_direct allow all
access_log /var/log/squid/access.log squid
```

On valide la syntaxe puis on redémarre Squid :

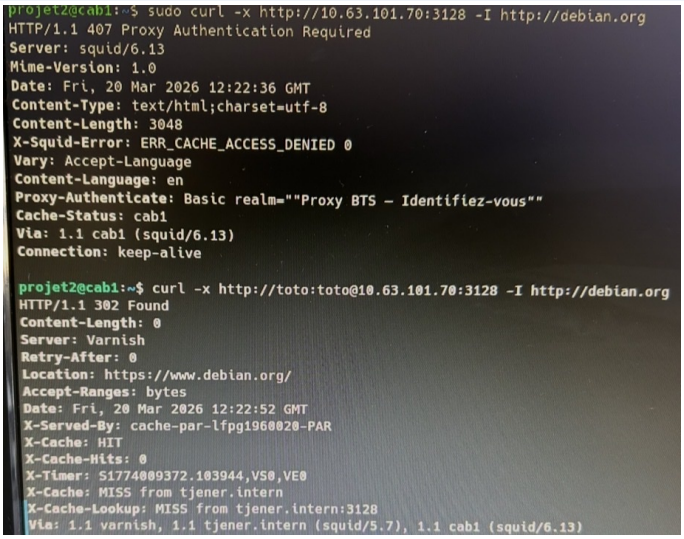
```
sudo squid -k parse 2>&1
sudo systemctl restart squid
sudo systemctl status squid
```

8. Tests de validation

Tous les tests sont réalisés avec curl -x depuis le conteneur cab1. Le code 407 prouve que l'authentification est obligatoire, 403 que le filtrage fonctionne, 301/302 que les sites autorisés sont accessibles.

```
sudo curl -x http://10.63.101.70:3128 -I http://debian.org
```

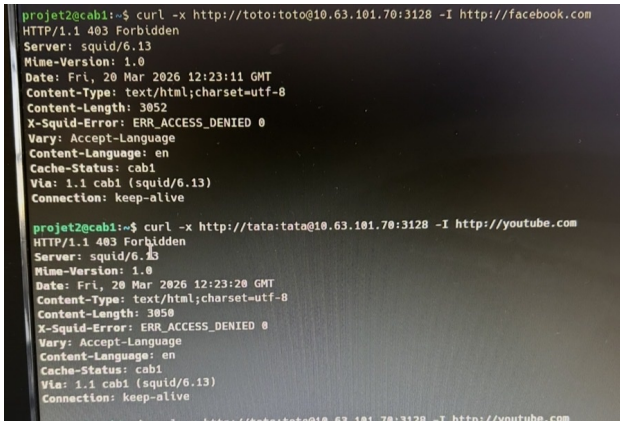
```
curl -x http://toto:toto@10.63.101.70:3128 -I http://debian.org
```



Sans auth → 407 Proxy Authentication Required / toto sur debian.org → 302 Found (autorisé — Via: cab1 + tjener.intern visible)

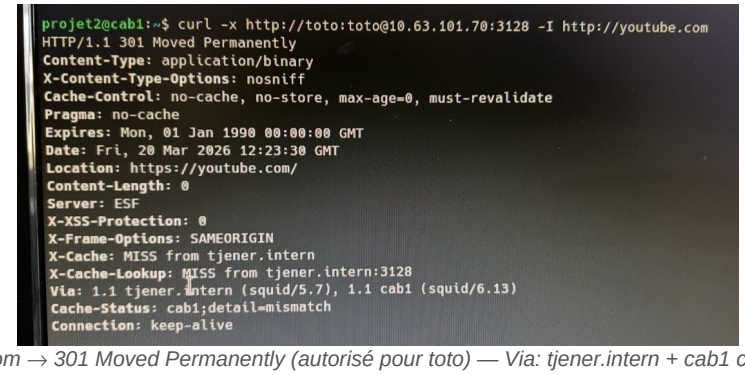
```
curl -x http://toto:toto@10.63.101.70:3128 -I http://facebook.com
```

```
curl -x http://tata:tata@10.63.101.70:3128 -I http://youtube.com
```



toto sur facebook.com → 403 Forbidden (bloqué pour tous) / tata sur youtube.com → 403 Forbidden (bloqué pour tata uniquement)

```
curl -x http://toto:toto@10.63.101.70:3128 -I http://youtube.com
```

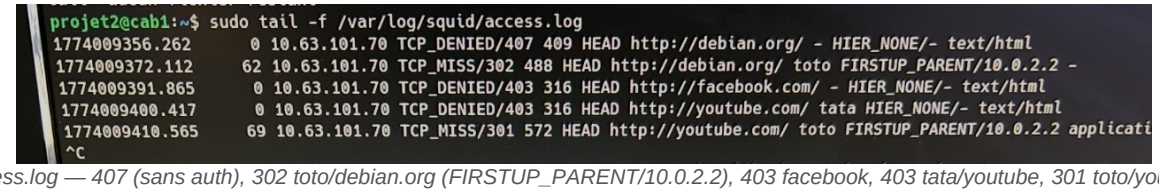


toto sur youtube.com → 301 Moved Permanently (autorisé pour toto) — Via: tjener.intern + cab1 confirme le chaînage

9. Analyse des logs Squid

Le fichier /var/log/squid/access.log enregistre toutes les requêtes. On y voit l'IP source, le code HTTP, l'URL et l'identifiant de l'utilisateur.

```
sudo tail -f /var/log/squid/access.log
```

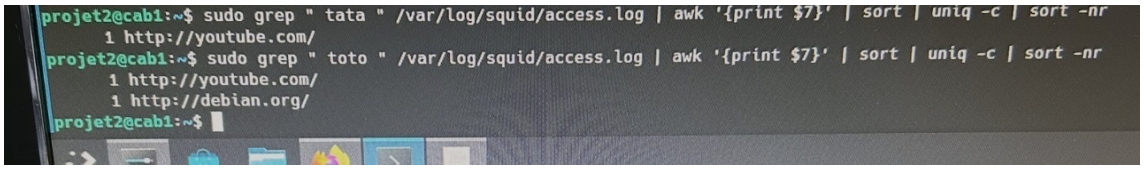


access.log — 407 (sans auth), 302 toto/debian.org (FIRSTUP_PARENT/10.0.2.2), 403 facebook, 403 tata/youtube, 301 toto/youtube

10. Comparaison des accès par utilisateur

La commande grep + awk extrait les URLs visitées par chaque utilisateur et permet de comparer leurs historiques d'accès.

```
sudo grep " tata " /var/log/squid/access.log | awk '{print $7}' | sort | uniq -c | sort -nr
sudo grep " toto " /var/log/squid/access.log | awk '{print $7}' | sort | uniq -c | sort -nr
```



grep tata → youtube.com (tentative bloquée) / grep toto → youtube.com + debian.org (accès autorisés)

11. Tableau récapitulatif des droits

Utilisateur	Sans auth	facebook / insta / tiktok	youtube / twitch	debian.org
Aucun	407 Refusé	—	—	—
toto	—	403 Bloqué	301 Autorisé	302 Autorisé
tata	—	403 Bloqué	403 Bloqué	302 Autorisé

12. Conclusion

Le proxy filtrant Squid est pleinement opérationnel dans le conteneur cab1 (10.63.101.70:3128). L'authentification Basic via htpasswd est effective — 407 sans identifiants, accès différencié selon le profil utilisateur. Le chaînage vers le Tjener (10.0.2.2) est confirmé par les en-têtes Via et par la colonne FIRSTUP_PARENT/10.0.2.2 dans les logs.

Limite identifiée : Squid ne filtre pas le trafic HTTPS par défaut car il est chiffré de bout en bout. La solution serait le SSL Bump (interception SSL avec certificat propre) — perspective d'amélioration hors périmètre de ce projet.